

Руководство по настройке отладчика в Qt Creator для отображения Call Stack и локальных переменных

Актуально для версии QT 4.x, на других работоспособность не гарантируется.

Введение



Для отладки в CentOS нужно обновить GDB до версии 8, добавить в "Настройки отладчика" Qt Creator и, затем, в комплект.

У всех, кто выполнял отладку в Qt Creator на **CentOS7** с использованием **Python 2.7**, скорее всего, возникала проблема, когда Call Stack не выводится, а чтобы отобразить локальные переменные, нужно на каждом шаге отладчика переключать флаги "Использовать помощников отладчика" или "Отображать реальный тип объекта".

Давно известно, что проблема эта возникает из-за символов кириллицы в путях к отлаживаемым библиотекам, а если подумать получше — то из-за кривой реализации прослойки между Qt Creator и GDB.

Для решения описанной проблемы следуйте приведённой далее инструкции.

Инструкция по настройке отладчика



ПРЕДУПРЕЖДЕНИЕ!

Все действия осуществляются на свой страх и риск. Делайте backup редактируемых файлов. Внимательно проверяйте все изменения!

1. Найдите файл gdbbridge.py в папке Qt Creator.

- Для начала проверьте основные пути, по которым может располагаться искомый файл:

```
1 /opt/qtcreator-X.X.X/share/qtcreator/debugger/gdbbridge.py
2 /opt/qtcreator/share/qtcreator/debugger/gdbbridge.py
3 /usr/share/qtcreator/debugger/gdbbridge.py
```

- Если нужного файла не нашлось по этим путям, тогда выполните команду в Терминале:

```
cd $(dirname $(which qtcreator))/../share/qtcreator/debugger
```

- Если предыдущая команда завершилась ошибкой, выполните другую команду:

```
cd $(dirname $(readlink $(which qtcreator)))/../share/qtcreator/debugger
```

Так выполняется перемещение в папку с файлом gdbbridge.py.

- Если предыдущие действия не помогли, найдите файл командой (запускать с правами суперпользователя):

```
find /usr /opt -name "gdbbridge.py"
```

2. Выполните команду backup gdbbridge.py (с правами суперпользователя):

```
cp gdbbridge.py gdbbridge.py.backup
```

3. Откройте gdbbridge.py любым редактором (с правами суперпользователя). Найдите в файле строку:

```
objfile = fromNativePath(symtab.objfile.filename)
```

и замените ее на:

```
1 try:
2     objfile = fromNativePath(symtab.objfile.filename)
3 except:
4     objfile = ''
```

В данном случае мы позволяем методу нормально отработать, вместо того, чтобы свалиться при попытке обращения к `symtab.objfile.filename`.

После вышеописанных действий локальные переменные будут обновляться автоматически после каждого шага GDB (как, собственно, и должно быть).

Инструкция по настройке отладчика для работы флага "Помощники отладчика"

Если необходимо, чтобы для некоторых типов работал флаг "Помощники отладчика", то есть, нормально отображались значения многих стандартных (и не только) типов (чтобы не приходилось раскрывать структуру переменной ручками), нужно выполнить следующее:

1. В файле `gdbbridge.py` найдите функцию `qtNamespace`:

```
1 def qtNamespace(self):
2     self.preping('qtNamespace')
3     res = self.qtNamespaceX()
4     self.ping('qtNamespace')
5     return res
```

Если функция выглядит так, то перейдите в п.2, иначе — в п.3.

2. Оберните содержимое функции в try-исхепт. Добавьте возврат пустой строки в случае Exception. В результате должно получиться подобное:

```
1 def qtNamespace(self):
2     res = ''
3     try:
4         self.preping('qtNamespace')
5         res = self.qtNamespaceX()
6         self.ping('qtNamespace')
7     except:
8         pass
9     return res
```

3. Если функция `qtNamespace` выглядит иначе, например:

```
1 def qtNamespace(self):
2     if not self.currentQtNamespaceGuess is None:
3         return self.currentQtNamespaceGuess
4     # This only works when called from a valid frame.
5     try:
6         cand = "QArrayData::shared_null"
7         symbol = gdb.lookup_symbol(cand)[0]
```

```

8         if symbol:
9             ns = symbol.name[:-len(cand)]
10            self.qtNamespaceToReport = ns
11            self.qtNamespace = lambda: ns
12            return ns
13        except:
14            pass
15        ...

```

тогда выполните следующее:

1. переименуйте функцию `def qtNamespace(self)` в `def qtNamespaceX(self)`.
2. добавьте выше этой функции другую функцию:

```

1 def qtNamespace(self):
2     res = ''
3     try:
4         res = self.qtNamespaceX()
5     except:
6         pass
7     return res

```

Результат:

```

1 def qtNamespace(self):
2     res = ''
3     try:
4         res = self.qtNamespaceX()
5     except:
6         pass
7     return res
8
9 def qtNamespaceX(self):
10    if not self.currentQtNamespaceGuess is None:
11        return self.currentQtNamespaceGuess
12    # This only works when called from a valid frame.
13    try:
14        cand = "QArrayData::shared_null"
15        symbol = gdb.lookup_symbol(cand)[0]
16        ...

```



Примечание

В данном случае мы не даем методу, который генерирует "красивые значения", свалиться на ненужном нам фрагменте кода, который "падает" с ошибкой `UnicodeDecodeError`.

Однако это не решает проблему для всех типов, в этом файле `gdbbridge.py` есть места, где можно получить ошибку `UnicodeDecodeError`.